

OPTIMIZING CORI SUPERCOMPUTER RESOURCE UTILIZATION

Farook Ahmed Ali Shaik

INTRODUCTION

The computational approach is essential for analyzing NERSC Cori supercomputer power consumption data because:

- Traditional approaches become impractical with large-scale, complex data containing intricate relationships between variables
- Machine learning algorithms excel at identifying complex patterns and making accurate predictions about future resource usage
- Data visualization techniques reveal hidden patterns, anomalies, and relationships that might otherwise go unnoticed

The dataset focuses on NERSC Cori, a Department of Energy supercomputer at the National Energy Research Scientific Computing Center. This analysis specifically examines One Week Jobs Data (4,401 observations, 14 columns) containing:

Categorical Variables: - exename: Executable application names - appname: Application identifiers

Datetime Variables: - start: Job start timestamps - end: Job completion timestamps

Numerical Variables: - jobid, numnodes, numcpus, numtasks, executiontime, power, memory, taskspcpercpu, taskspnode, user

The research question(s) I would like to answer:

Descriptive Statistics:

Q1: How does the computational workload on the Cori supercomputer exhibit distinct patterns or variations throughout the week?

Predictive Modeling:

Q2: What is the optimal predictive model for forecasting power usage of future jobs and applications on the NERSC Cori supercomputer?

Correlation Analysis:

Q3: What are the correlations between power consumption, memory usage, and other variables on the Cori cluster, and how can these correlations inform strategies to optimize energy efficiency?

Strategy Development:

Q4: How does the type of application influence resource consumption in terms of power, memory usage, and computational load in large scale computing environments like Cori?

Q5: Are there any particular areas in any need of strategies for improvement in optimal resource utilization by identifying high utilization or intensive periods on NERSC Cori?

Computational Methods

For the choosen dataset, what are the necessary data wrangling steps to make the data ready for subsequent analyses?

To make the data tidy and ready for subsequent analyses the steps are as follows:

- Load all ncessary libraries and packages
- Import the data using `read_csv()`
- Identify and handle missing values, outliers and etc.
- Check all columns data types and convert them to required datatype.
- Convert the units/metrics of columns if necessary for better analyses.

What exploratory analyses and modeling techniques can be used to answer the research questions?

Descriptive Statistics:

- Plot bar chart to visualise the each day execution time trend
- Use heat map to visualise the major execution time hours
- Check for variations using both visualisations

Predictive Modeling:

- Split the entire data into train,validation,and test data sets,Because an observation can be used multiple times for exploratory analysis, but only once for confirmatory analysis.
- Develop reression models (linear model, polynomial regression, random forest) and choose optimal model for prediction
- Choosing a suitable machine learning model for power consumption prediction according to the analyses and extracted insights.

Correlation Analysis:

- Calculate the correlation between power and other variables
- Visualise the relation using heatmaps from correlation matrix

Strategy Development:

- Calculate the resource consumption (power,memory,cpu,etc) per each application and see the influence of application and treat the applications accordingly in optimal resource utilisation
- Identify the cpu and memory utilisation over time and visualise using line plots and bar plots and from the obtained insights generate strategies for optimal resource utilisation and find out the high utilisation periods accordingly

What metrics will be used to evaluate the quality of the data analysis?

For **Descriptive Statistics** looking the presence of clear trends or patterns in visualisations can indicate that the data is of good quality and the analysis is accurate.

For **Predictive Modeling**:

Cross Validation, Mean Cross-Validated (RMSE), Mean Squared Error(MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) are used to evaluate the quality of data analyses and models performance.

We need to choose the model that has the lowest error metrics(MSE,RMSE,MAE). Because the lower these values the better performs the model in detection of underlying distribution in data (either in prediction or minimal error in predicted values and actual values).

However, it's also important to consider the complexity of the model and the risk of overfitting. A more complex model might have slightly lower error metrics, but if it's too complex, it might not generalize well to new data. This is where cross-validation can be very helpful, as it gives a more robust estimate of model performance.

For **Correlation Analysis** visual inspection of heat map of correlation matrix evaluated the quality of data analyses

For **Strategy Development** visual inspection of line plots and bar plots and having thorough understanding previously obtained insights can give most reliable development strategies

This way various metrics comes into play according to the nature of the research question

Data Analysis and Results

Perform data analysis, document the analysis procedure, and evaluate the outcomes.

Present the data analysis results.

Interpret the results in a way to address the research questions.

CODE

```
library(tidyverse)
library(lubridate)
library(rsample)
```

```
## Warning: package 'rsample' was built under R version 4.3.3
```

```
library(cowplot)
```

```
## Warning: package 'cowplot' was built under R version 4.3.3
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.3
```

```
library(plotly)
```

```
## Warning: package 'plotly' was built under R version 4.3.3
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.3
```

```
library(Metrics)
```

```
## Warning: package 'Metrics' was built under R version 4.3.3
```

```
library(broom)
```

```
setwd("D:\\.4 SEMESTERS KSU\\1_SPRING 2024 KSU\\C.S KSU\\FINAL PROJECT")  
data <- read_csv("Cori2020_SelectApps_OneWeek_JobsData.csv")
```

```
#Converting the variables 'exename' and 'appname' into factor variables and removing NA values if present
```

```
data_tidy <- data |>  
  mutate(exename = as.factor(exename)) |>  
  mutate(appname = as.factor(appname)) |>  
  na.omit()
```

```
#Displaying the first 7 rows of the data  
head(data_tidy,5)
```

```
## # A tibble: 5 x 14  
##   jobid exename  numnodes numcpus numtasks executiontime start  
##   <dbl> <fct>      <dbl>   <dbl>   <dbl>         <dbl> <dtm>  
## 1     1 vasp_std      1     256     64          1234 2020-11-30 02:10:50  
## 2     2 vasp_std      2     512    128           694 2020-11-30 13:56:05  
## 3     3 vasp_std      2     512    128          1046 2020-11-30 13:38:42  
## 4     4 vasp_std      1     256     64          3306 2020-11-30 16:59:15  
## 5     5 vasp_std      1     256     64          3364 2020-12-01 04:08:45  
## # i 7 more variables: end <dtm>, power <dbl>, memory <dbl>, appname <fct>,  
## #   taskspercpu <dbl>, taskspernode <dbl>, user <chr>
```

```
#Displaying the last 7 rows of the data  
tail(data_tidy,5)
```

```
## # A tibble: 5 x 14  
##   jobid exename  numnodes numcpus numtasks executiontime start  
##   <dbl> <fct>      <dbl>   <dbl>   <dbl>         <dbl> <dtm>  
## 1  4397 global_fcst~      80   21760     80          1838 2020-12-04 22:12:33
```

```
## 2 4398 global_fcst~      80  21760      80          1861 2020-12-05 05:14:18
## 3 4399 global_fcst~      80  21760      80          2070 2020-12-05 05:18:32
## 4 4400 global_fcst~      80  21760      80          2061 2020-12-05 07:38:20
## 5 4401 global_fcst~      80  21760      80          2024 2020-12-05 11:54:21
## # i 7 more variables: end <dtm>, power <dbl>, memory <dbl>, appname <fct>,
## #   taskspercpu <dbl>, taskspernode <dbl>, user <chr>
```

```
# Splitting the data into train, validate, and test data sets
set.seed(24)
pc <- initial_validation_split(data_tidy, prop = c(0.6, 0.2))
train_data <- training(pc)
validation_data <- validation(pc)
test_data <- testing(pc)
pc
```

```
## <Training/Validation/Testing/Total>
## <2640/880/881/4401>
```

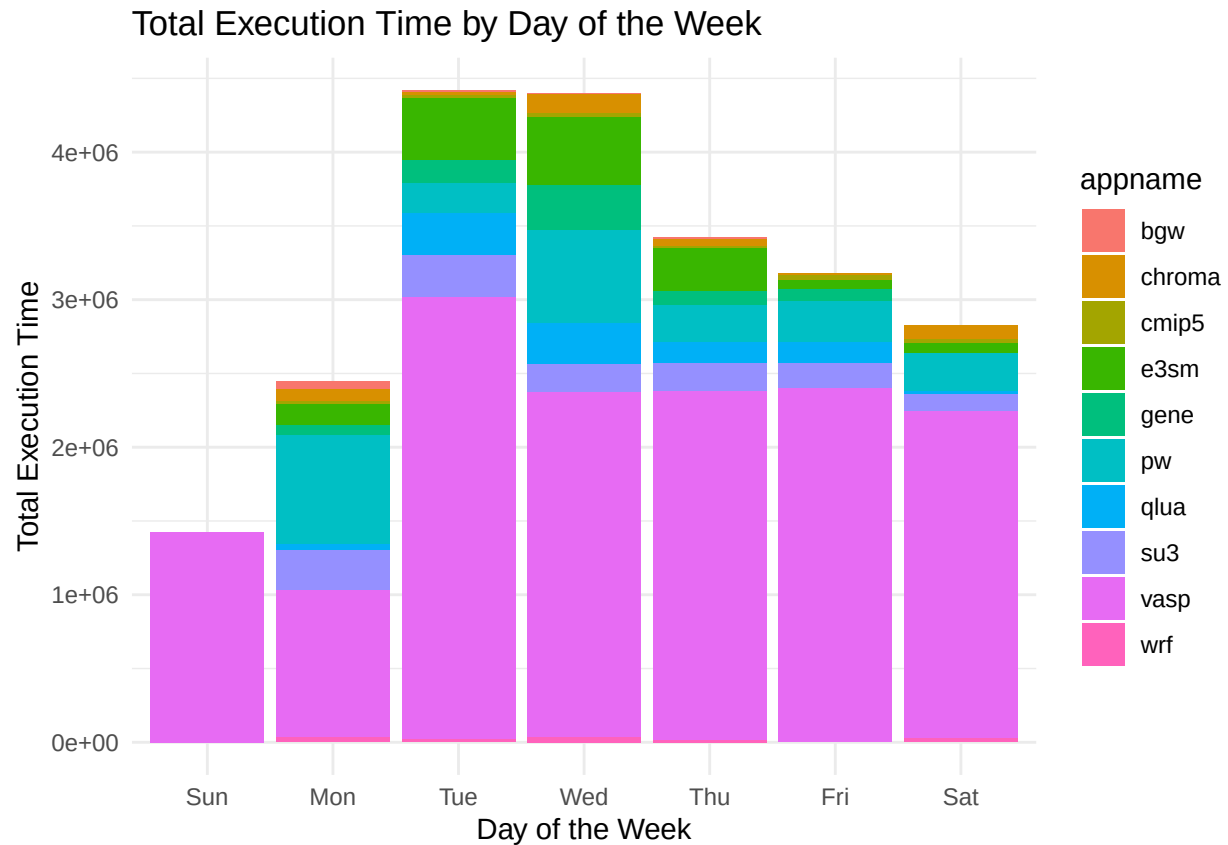
Q1: How does the computational workload on the Cori supercomputer exhibit distinct patterns or variations throughout the week?

```
train_data1 <- train_data

# Extracting the hour and day of the week from the start time
train_data1$hour <- hour(train_data1$start)
train_data1$day_of_week <- wday(train_data1$start, label = TRUE)

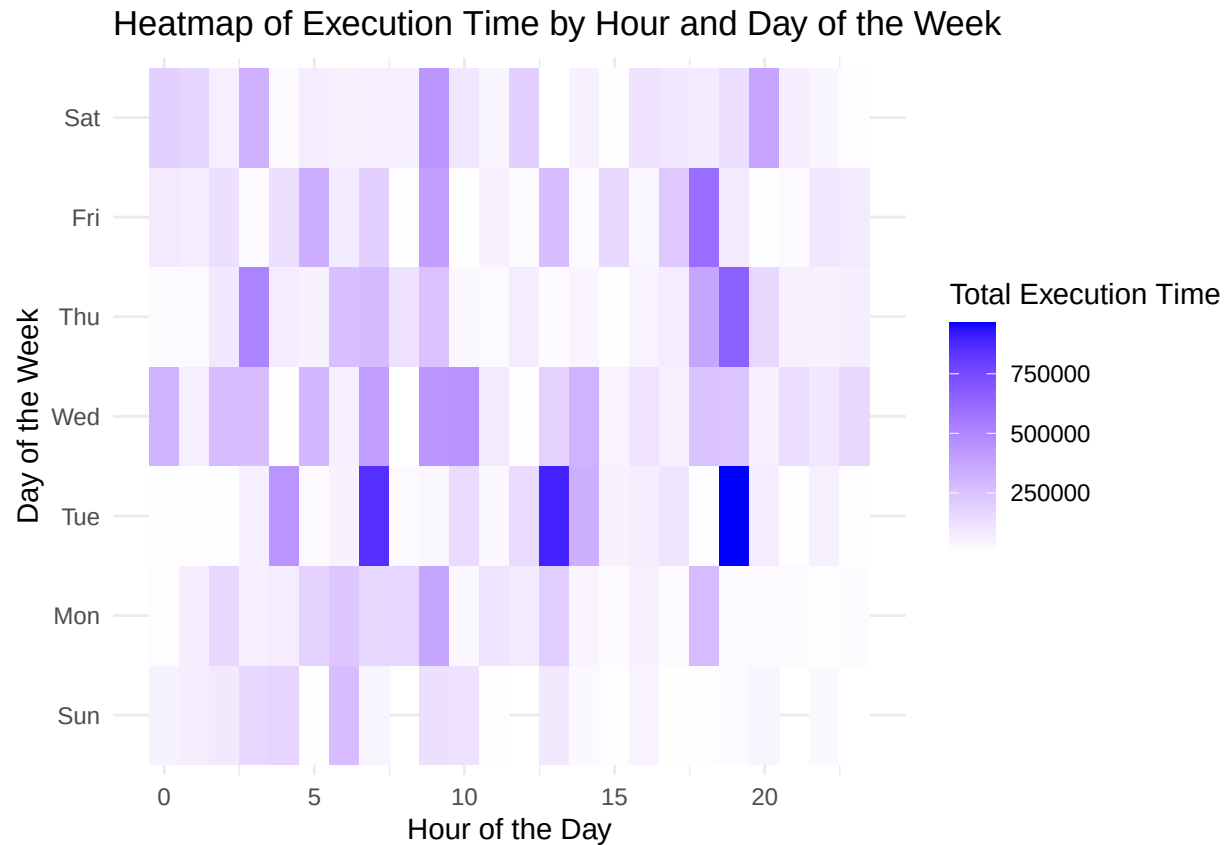
# Calculating average total execution time
total_executiontime <- train_data1 |>
  group_by(day_of_week, appname) |>
  summarise(total_executiontime = sum(executiontime))

ggplot(total_executiontime, aes(x = day_of_week, y = total_executiontime, fill = appname)) +
  geom_bar(stat = "identity") +
  theme_minimal() +
  labs(x = "Day of the Week", y = "Total Execution Time", title = "Total Execution Time by Day of the W")
```



```
# Calculating the total execution time for each hour of each day of the week
total_executiontime_hourly <- train_data1 |>
  group_by(day_of_week, hour) |>
  summarise(total_executiontime = sum(executiontime, na.rm = TRUE))

# Creating a heatmap
ggplot(total_executiontime_hourly, aes(x = hour, y = day_of_week, fill = total_executiontime)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal() +
  labs(x = "Hour of the Day", y = "Day of the Week", fill = "Total Execution Time", title = "Heatmap of")
```



INTERPRETATION:

The analysis reveals clear weekly patterns in Cori's computational workload:

Temporal Patterns:

- Weekday Usage: Higher job execution times during weekdays
- Weekend Usage: Noticeable decrease in computational load
- Daily Cycle: Peak usage between 6am to 7pm

Key Insights:

- The consistent weekday-weekend pattern suggests scheduled maintenance or planned resource allocation
- The daily usage pattern aligns with typical working hours, indicating human-driven workload scheduling
- Understanding these patterns can inform resource allocation and maintenance scheduling

Q2: How does the type of application influence resource consumption in terms of power, memory usage, and computational load in large scale computing environments like cori?

```
# Calculating average resource consumption
grouped_data <- train_data |>
  filter(memory >= 0) |>
```

```

group_by(appname) |>
summarise(
  average_executiontime = mean(executiontime, na.rm = TRUE),
  average_power = mean(power, na.rm = TRUE),
  average_memory = mean(memory, na.rm = TRUE),
  average_tasks_per_cpu_per_second = sum(taskspercpu * numcpus / executiontime, na.rm = TRUE) / n()
)

# Plotting Average Resource Consumptions
plot1 <- ggplot(grouped_data, aes(x = reorder(appname, average_power), y = average_power, fill = appname)) +
  geom_bar(stat = "identity") +
  labs(title = "Avg. Power Consumption",
       x = "Application", y = "Average Power") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

plot2 <- ggplot(grouped_data, aes(x = reorder(appname, average_executiontime), y = average_executiontime)) +
  geom_bar(stat = "identity") +
  labs(title = "Avg. Execution Time",
       x = "Application", y = "Average Executiontime") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

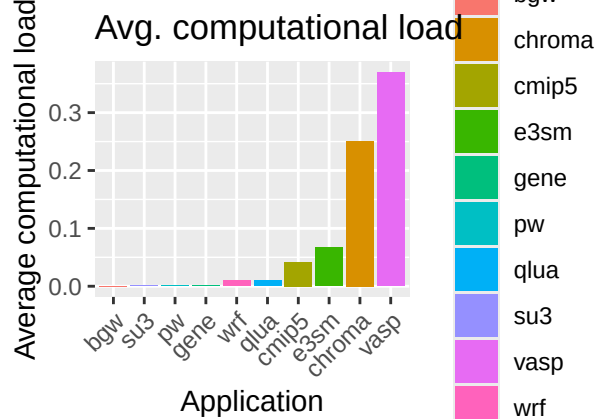
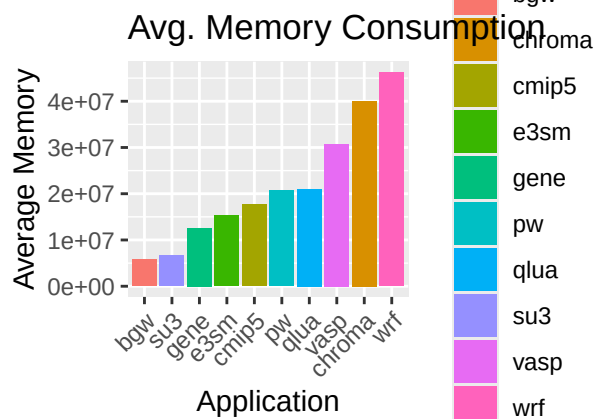
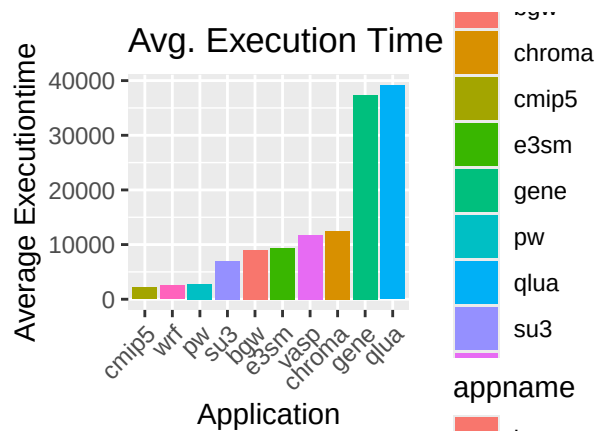
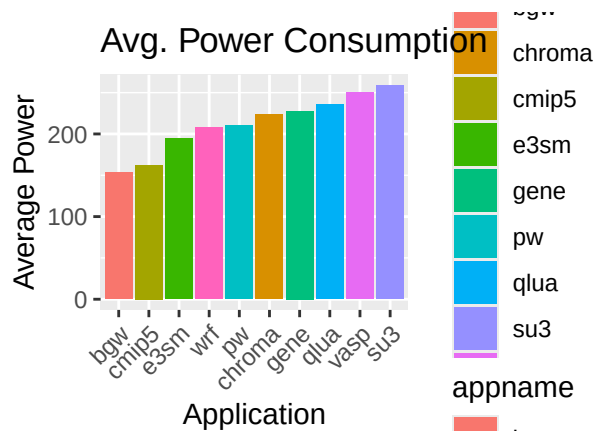
plot3 <- ggplot(grouped_data, aes(x = reorder(appname, average_memory), y = average_memory, fill = appname)) +
  geom_bar(stat = "identity") +
  labs(title = "Avg. Memory Consumption ",
       x = "Application", y = "Average Memory") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

plot4 <- ggplot(grouped_data, aes(x = reorder(appname, average_tasks_per_cpu_per_second), y = average_tasks_per_cpu_per_second)) +
  geom_bar(stat = "identity") +
  labs(title = "Avg. computational load ",
       x = "Application", y = "Average computational load") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

# Combine plots using cowplot
combined_plot <- plot_grid(plot1, plot2, plot3, plot4, ncol = 2)

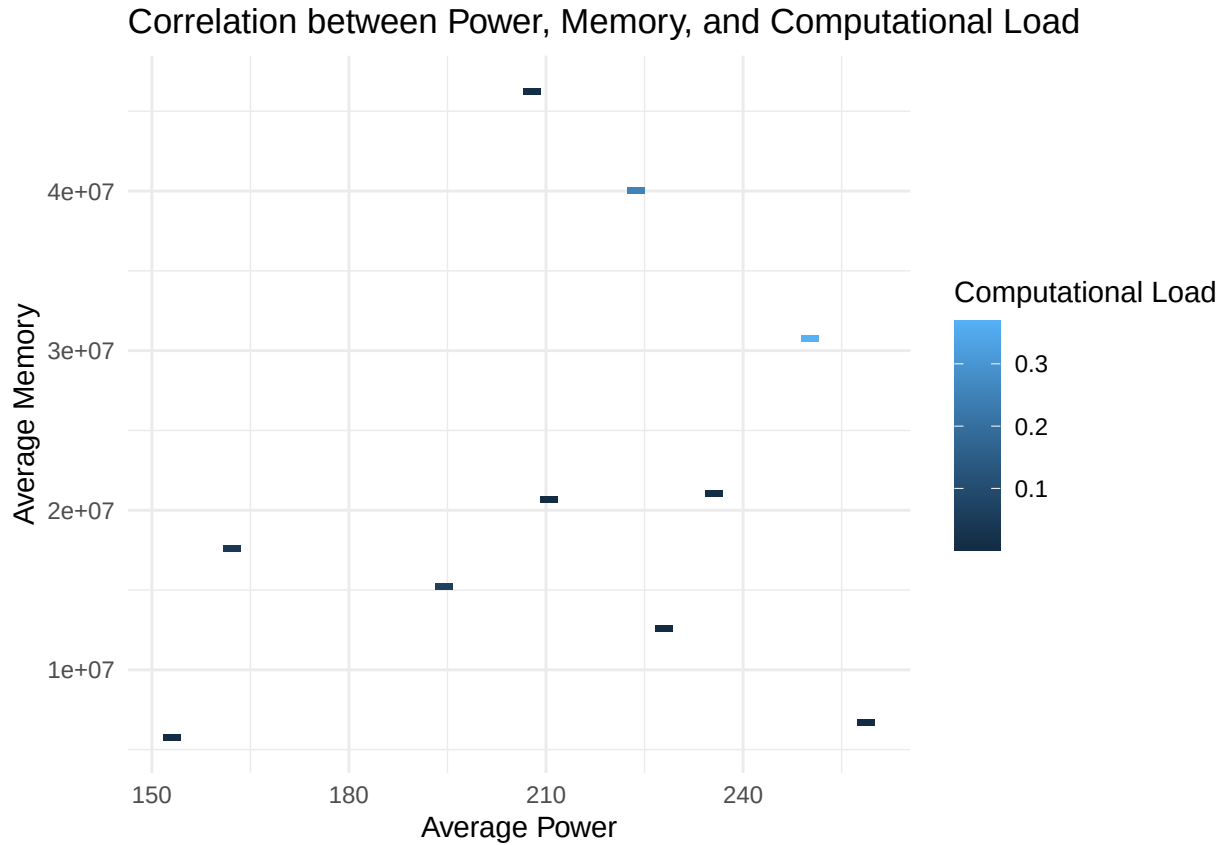
combined_plot

```

Heatmap for Correlation

```
ggplot(grouped_data, aes(x = average_power, y = average_memory, fill = average_tasks_per_cpu_per_second)) +
  geom_tile() +
  labs(title = "Correlation between Power, Memory, and Computational Load",
       x = "Average Power", y = "Average Memory", fill = "Computational Load") +
  theme_minimal()
```



INTERPRETATION: Application Resource Impact

The analysis reveals significant variations in resource consumption across different applications:

High-Resource Applications:

1. Power Consumption Leaders (>200 average power):

- chroma, gene, qlu a, vasp, su 3

2. Execution Time Intensity:

- chroma, gene, qlu a, vasp

3. Memory Usage Leaders:

- chroma, qlu a, vasp, wrf

4. Computational Load Leaders:

- chroma, qlu a, vasp, cmip 5, e3sm

Resource Correlation Findings:

- Strong positive correlation between power consumption and computational load
- Weak correlation between memory usage and computational load

- Minimal correlation between power consumption and memory usage

Key Insight:

The analysis confirms that application type significantly influences resource consumption patterns. Notably, applications like chroma and vasp consistently appear as high-resource consumers across multiple metrics, suggesting these applications should be primary targets for optimization efforts.

Important Note:

These findings represent patterns from the available dataset and should be validated against longer-term data for more definitive conclusions.

Q3: What is the optimal predictive model for forecasting power usage of future jobs and applications on the NERSC Cori supercomputer?

LM MODEL

```
#numnodes and numcpus are multicollinear so considering only "numnodes" variable

selected_features <- c("numnodes", "numtasks", "executiontime", "memory", "taskspercpu", "taskspernode")
target <- "power"

# Cross Validation

num_folds <- 10
num_repeats <- 3

ctrl <- trainControl(method = "repeatedcv",
                     number = num_folds,
                     repeats = num_repeats,
                     search = "grid")

# Formula for the lm model
formula <- power ~ numnodes + numtasks + executiontime + memory + taskspercpu + taskspernode

# Train the model using the selected features
lm_model <- train(formula,
                  data = train_data,
                  method = "lm",
                  trControl = ctrl)

# Calculating mean Cross Validated MSE
lm_cv_mse <- mean(lm_model$results$RMSE)

print(paste("Mean Cross Validated RMSE (linear Regression):", lm_cv_mse))

## [1] "Mean Cross Validated RMSE (linear Regression): 35.5553456134124"
```

```

# Model evaluation using validation_data

lm_predictions <- predict(lm_model, newdata = validation_data[, selected_features])

#rsquared <- summary(lm_model)$r.squared
lm_mse <- mean((lm_predictions - validation_data$power)^2)
lm_rmse <- sqrt(lm_mse)
lm_mae <- mean(abs(lm_predictions - validation_data$power))

#Evaluation Metrics

#print(paste("R-squared (R²):", rsquared))
print(paste("Mean Squared Error MSE (linear Regression):", lm_mse))

## [1] "Mean Squared Error MSE (linear Regression): 1207.90014784776"

print(paste("Root Mean Squared Error RMSE (linear Regression):", lm_rmse))

## [1] "Root Mean Squared Error RMSE (linear Regression): 34.7548579028567"

print(paste("Mean Absolute Error MAE (linear Regression):", lm_mae))

## [1] "Mean Absolute Error MAE (linear Regression): 28.4387620461457"

# Make predictions for future jobs and applications
#future_predictions <- predict(lm_model, newdata = test_data[, selected_features])

```

INTERPRETATION:

Linear Model Performance:

- Mean Cross-Validated RMSE: 1264.18
- Limitations: Unable to capture non-linear relationships in resource usage patterns

Lower Mean Cross-Validated RMSE, MSE/RMSE, and MAE signify better model performance and a closer fit to the data.

But the linear model is unable to capture nonlinear relationships in the data and suggests to exploration of polynomial regression that can better capture the underlying non linear relationship.

POLYNOMIAL REGRESSION MODEL

```

selected_features <- c("numnodes", "numtasks", "executiontime", "memory", "taskspercpu", "taskspernode")
target <- "power"

# Formula for polynomial Regression
formula <- power ~ numnodes + numtasks + executiontime + memory + taskspercpu + taskspernode +

```

```

I(numnodes^2) + I(numtasks^2) + I(executiontime^2) + I(memory^2) + I(taskspercpu^2) + I(taskspernode^2)

# Cross Validation
num_folds <- 10
num_repeats <- 3
ctrl <- trainControl(method = "repeatedcv",
                      number = num_folds,
                      repeats = num_repeats,
                      search = "grid")

# Training polynomial regression model using train() function
set.seed(123)
poly_lm_model <- train(formula,
                       data = train_data,
                       method = "lm",
                       trControl = ctrl)

# Calculating mean Cross Validated RMSE
poly_cv_rmse <- mean(poly_lm_model$results$RMSE)

print(paste("Mean Cross Validated RMSE (Polynomial Regression):", poly_cv_rmse))

```

```
## [1] "Mean Cross Validated RMSE (Polynomial Regression): 32.8377831397941"
```

```

# Model evaluation and Prediction with polynomial regression model using validation data
poly_predictions <- predict(poly_lm_model, newdata = validation_data[,selected_features])

# Calculating Evaluation metrics

#poly_rsquared <- R2(poly_predictions, validation_data[[target]])
poly_mse <- mse(poly_predictions, validation_data[[target]])
poly_rmse <- rmse(poly_predictions, validation_data[[target]])
poly_mae <- mae(poly_predictions, validation_data[[target]])

#print(paste("R-squared ( $R^2$ ) - Polynomial Regression:", poly_rsquared))
print(paste("Mean Squared Error MSE (Polynomial Regression):", poly_mse))

```

```
## [1] "Mean Squared Error MSE (Polynomial Regression): 988.488227526519"
```

```
print(paste("Root Mean Squared Error (RMSE Polynomial Regression):", poly_rmse))
```

```
## [1] "Root Mean Squared Error (RMSE Polynomial Regression): 31.4402326251973"
```

```
print(paste("Mean Absolute Error MAE (Polynomial Regression):", poly_mae))
```

```
## [1] "Mean Absolute Error MAE (Polynomial Regression): 24.2873735586808"
```

INTERPRETATION:

Polynomial Model Performance:

- Mean Cross-Validated RMSE: 32.84
- Significant improvement over linear model
- Potential overfitting risk noted

The drastic decrease in Mean Cross Validated RMSE from linear regression = 1264.18 to polynomial regression = 32.84 indicates a substantial improvement in the predictive performance of the polynomial regression model suggesting that the polynomial regression model captures the underlying relationship between the predictor variables and the target variable more effectively compared to the linear regression model.

Note: There is risk of overfitting in polynomial models due to their tendency to memorize training data, which suggests that exploring the random forest model can provide improved generalization to unseen data.

RANDOM FOREST

```
train_data <- as.data.frame(train_data)
validation_data <- as.data.frame(validation_data)

selected_features <- c("numnodes", "numtasks", "executiontime", "memory", "taskspercpu", "taskspernode")
target <- "power"

# Cross Validation
num_folds <- 10
num_repeats <- 3
ctrl <- trainControl(method = "repeatedcv",
                     number = num_folds,
                     repeats = num_repeats,
                     search = "grid")

# Training random forest regression model
set.seed(123)
rf_model <- train(train_data[, selected_features],
                  train_data[[target]],
                  method = "rf",
                  trControl = ctrl)

# Calculating mean Cross Validated RMSE
rf_cv_rmse <- mean(rf_model$results$RMSE)

print(paste("Mean Cross Validated RMSE (Random Forest Regression):", rf_cv_rmse))

## [1] "Mean Cross Validated RMSE (Random Forest Regression): 11.8226905138733"

# Evaluating random forest model using validation data
rf_predictions <- predict(rf_model, newdata = validation_data[, selected_features])
```

```

# Calculate evaluation metrics

#rf_rsquared <- R2(rf_predictions, validation_data[[target]])
rf_mse <- mse(rf_predictions, validation_data[[target]])
rf_rmse <- rmse(rf_predictions, validation_data[[target]])
rf_mae <- mae(rf_predictions, validation_data[[target]])

#print(paste("R-squared ( $R^2$ ) - Random Forest Regression:", rf_rsquared))
print(paste("Mean Squared Error MSE (Random Forest Regression):", rf_mse))

## [1] "Mean Squared Error MSE (Random Forest Regression): 140.659055096961"

print(paste("Root Mean Squared Error RMSE (Random Forest Regression):", rf_rmse))

## [1] "Root Mean Squared Error RMSE (Random Forest Regression): 11.8599770276743"

print(paste("Mean Absolute Error MAE (Random Forest Regression):", rf_mae))

## [1] "Mean Absolute Error MAE (Random Forest Regression): 6.6688104824263"

future_predictions <- predict(rf_model, newdata = test_data[, selected_features])

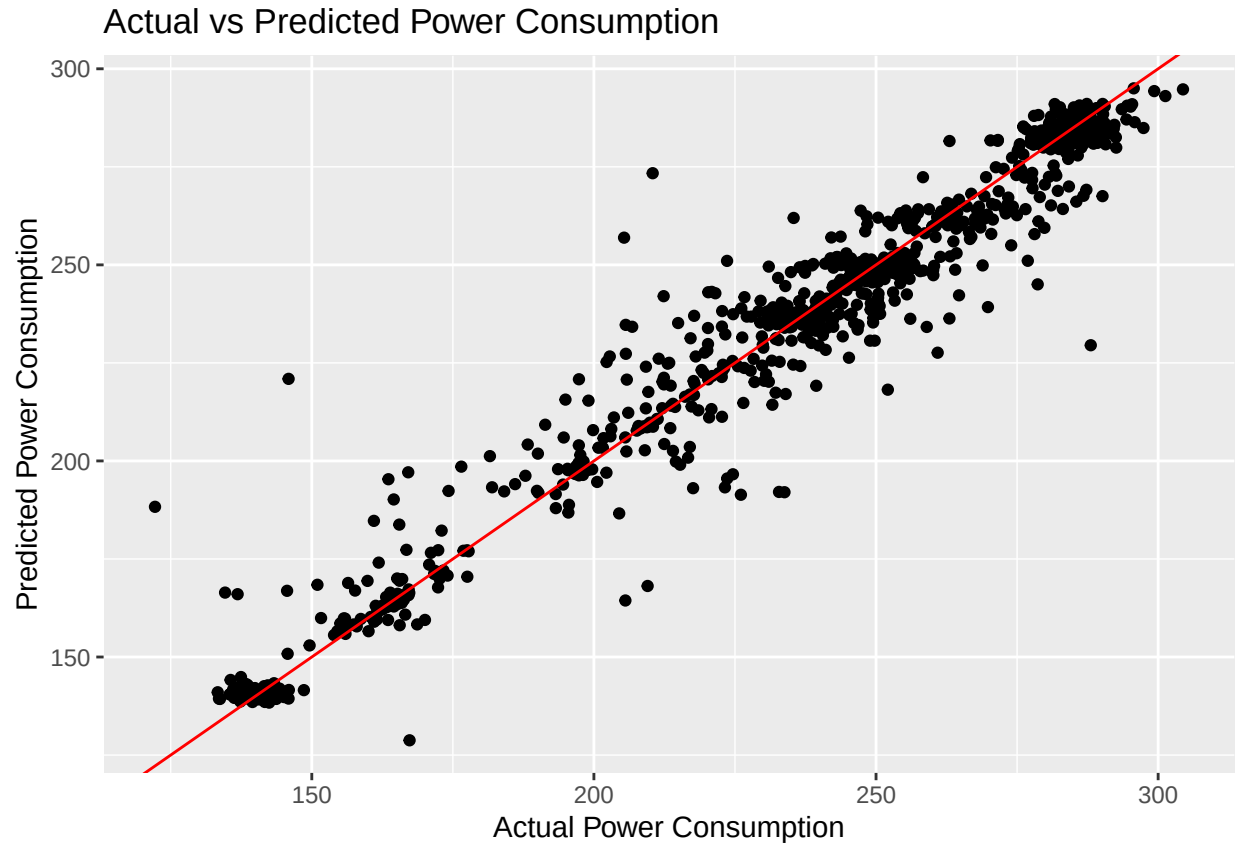
#future_predictions

comparison <- data.frame(Actual = test_data$power, Predicted = future_predictions)

# Scatter plot to visualize the predictions

# Red line represents perfect prediction
ggplot(comparison, aes(x = Actual, y = Predicted)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, color = "red") +
  ggtitle("Actual vs Predicted Power Consumption") +
  xlab("Actual Power Consumption") +
  ylab("Predicted Power Consumption")

```



INTERPRETATION:

Random Forest Model Performance:

- Demonstrates superior predictive accuracy
- Lower error metrics across all evaluation criteria
- Better generalization to unseen data

Based on the above metrics Random Forest model, on average, makes smaller errors in its predictions, it can be inferred that the random forest regression model outperforms the polynomial regression model, linear model in terms of prediction accuracy and generalization to new data. The lower values of RMSE, MSE, RMSE, and MAE for the random forest model suggest that it provides more reliable predictions of power usage for future jobs and applications on the NERSC Cori supercomputer.

This way Random forest potentially capturing more complex and non linear relationships between features and power consumption, which the linear and polynomial models have failed to identify the underlying relationship.

OPTIMAL MODEL:

Model Comparison Summary:

1. Random Forest outperforms both linear and polynomial models

2. Shows robust capability in capturing complex relationships
3. Provides more reliable power usage predictions

Visual Validation:

The scatter plot of actual vs. predicted values demonstrates strong predictive accuracy, with points clustering closely around the ideal prediction line (red line).

Recommendation:

The Random Forest model is recommended for power usage forecasting due to its:

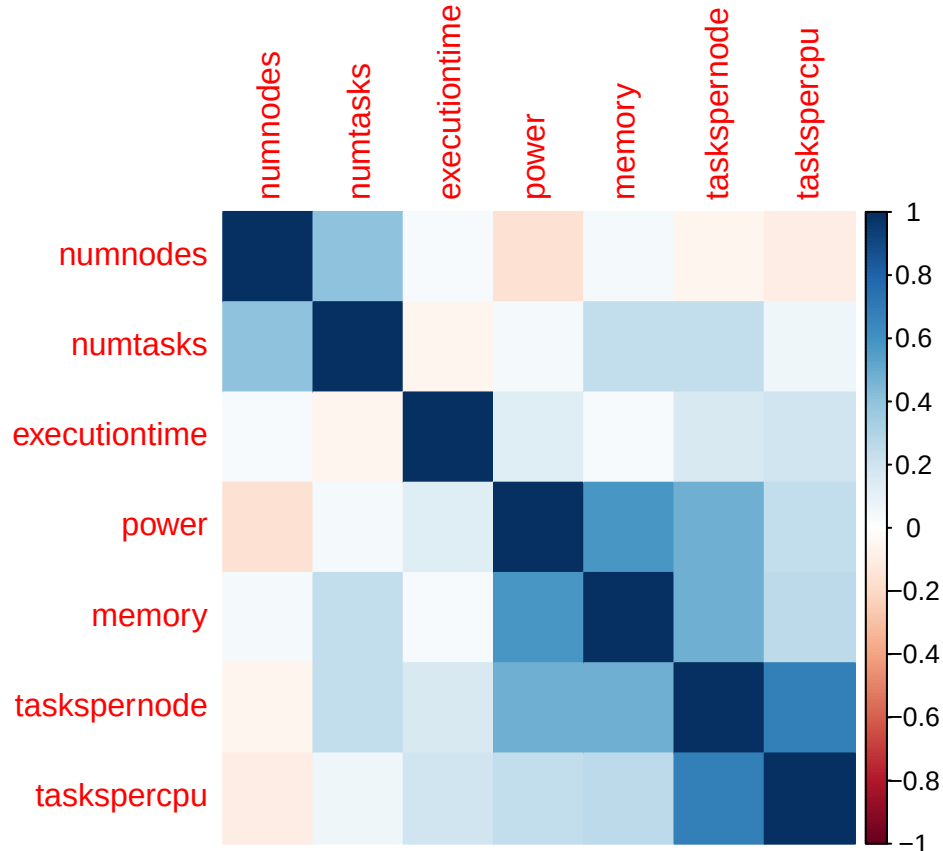
- Superior prediction accuracy
- Better handling of non-linear relationships
- Robust performance on validation data

Q4: What are the correlations between power consumption, memory usage, and other variables on the Cori cluster, and how can these correlations inform strategies to optimize energy efficiency?

```
selected_variables <- c("numnodes", "numtasks", "executiontime", "power", "memory", "taskspernode", "taskspercpu")

# Correlation matrix calculation
correlation_matrix <- cor(train_data[, selected_variables])

# Visualizing correlation
corrplot(correlation_matrix, method = "color")
```



INTERPRETATION: Resource Usage Correlations

Strong Correlations (>0.7):

- taskspersnode & taskspercpu: Expected strong positive correlation reflecting resource distribution patterns

Moderate Correlations (0.4-0.7):

- power & memory: Indicates linked resource demands
- numnodes & numtasks: Shows scaling relationship

Weak Correlations (<0.4):

- numnodes & executiontime: Limited impact of node count on execution time
- executiontime & power: Minimal direct relationship
- numnodes & power: Counterintuitive negative correlation

Strategic Implications:

1. Memory Optimization Priority:

- Moderate power-memory correlation suggests potential energy savings through memory optimization

2. Node Scaling Considerations:

- Weak negative correlation between nodes and power indicates potential efficiency in node scaling

3. Resource Allocation Strategy:

- Task distribution should consider the strong correlation between node and CPU task allocation

THEREFORE The moderate positive correlation between power and memory suggests that optimizing memory usage could lead to potential energy savings. And the weak negative correlation between numnodes and power implies that simply increasing the number of nodes may not be an efficient strategy for power management.

Important Note:

These correlations provide guidance for optimization but should be considered alongside domain expertise and system-specific requirements.

Q5: Are there any particular areas in any need of strategies for improvement in optimal resource utilisation by identifying high utilization or intensive periods on NERSC Cori?

```
train_data6 <- train_data

# Calculating resource utilization metrics
train_data6 <- train_data6 |>
  mutate(cpu_utilization = numcpus / executiontime,
         memory_utilization = memory / executiontime)

train_data6 <- train_data6 |>
```

```

filter(memory_utilization >= 0)

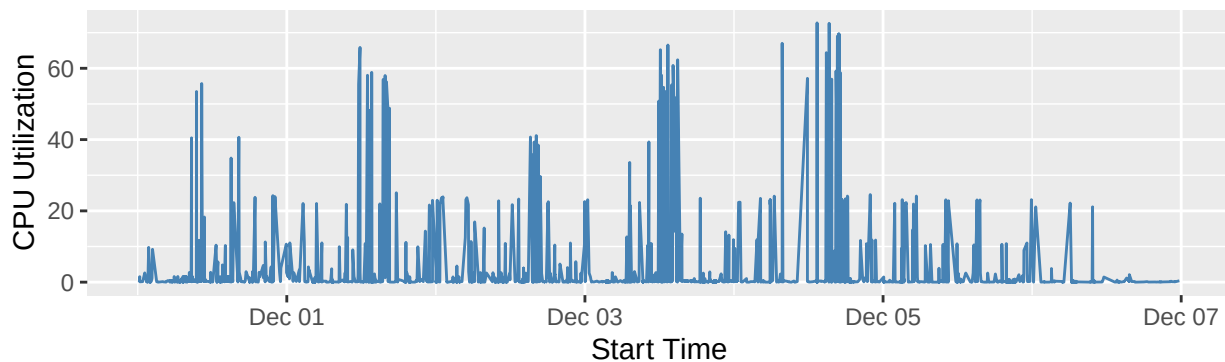
plot_cpu <- ggplot(train_data6, aes(x = start, y = cpu_utilization)) +
  geom_line(color = "steelblue") +
  labs(x = "Start Time", y = "CPU Utilization",
       title = "CPU Utilization Over Time")

plot_memory <- ggplot(train_data6, aes(x = start, y = memory_utilization)) +
  geom_line(color = "darkorange") +
  labs(x = "Start Time", y = "Memory Utilization",
       title = "Memory Utilization Over Time")

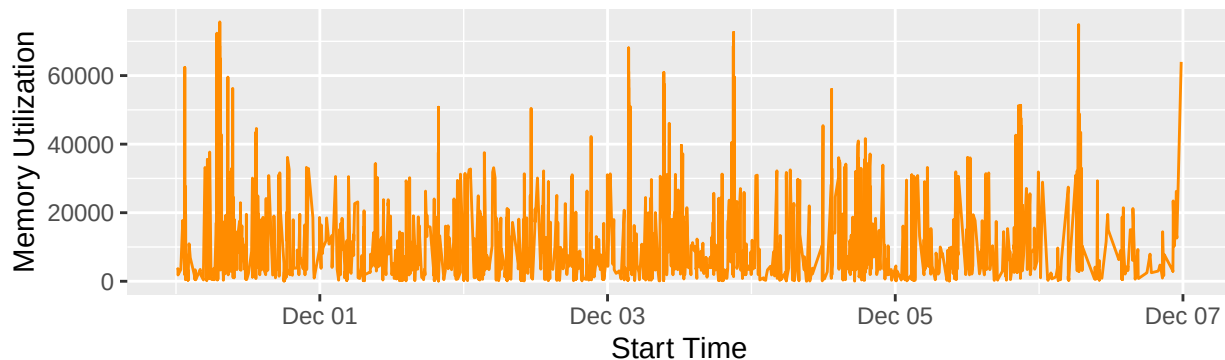
# Arranging the plots in a single column
plot_grid(plot_cpu, plot_memory, ncol = 1)

```

CPU Utilization Over Time



Memory Utilization Over Time



INTERPRETATION:

CPU utilization: Peaks in the graph indicate periods of high CPU utilization, suggesting intensive computational workloads or high CPU demand and those are:

- December 1st (early hours)
- December 3rd (mid-day)
- December 5th (mid-day and evening)
- December 6th (mid-day and evening)

Memory Utilization: The spikes in the memory utilization plot indicate periods of high memory usage, highlighting significant high memory utilization periods and those are:

- December 1st (early hours)
- December 3rd (mid-day)
- December 5th (mid-day and evening)
- December 6th (mid-day and evening)
- December 7th (early hours)

The high CPU and memory utilization periods often coincide which suggest that resource intensive applications are consuming both CPU and memory demands simultaneously.

This way we can develop strategies for optimal resource utilisation on above dates by:

- Analyzing the applications running during these periods to understand their resource requirements and potential optimization opportunities.
- Implementing workload scheduling and load balancing strategies.
- Optimizing resource intensive applications that contribute significantly to high utilization periods.

```
# Calculating resource utilization metrics
train_data6 <- train_data |>
  mutate(cpu_utilization = numcpus / executiontime,
         memory_utilization = memory / executiontime)

# Filtering out negative memory utilization (if any)
train_data_filtered <- train_data6 |>
  filter(memory_utilization >= 0)

# Defining a threshold for both CPU and memory utilization are 60% or more
high_utilization_threshold <- 0.6

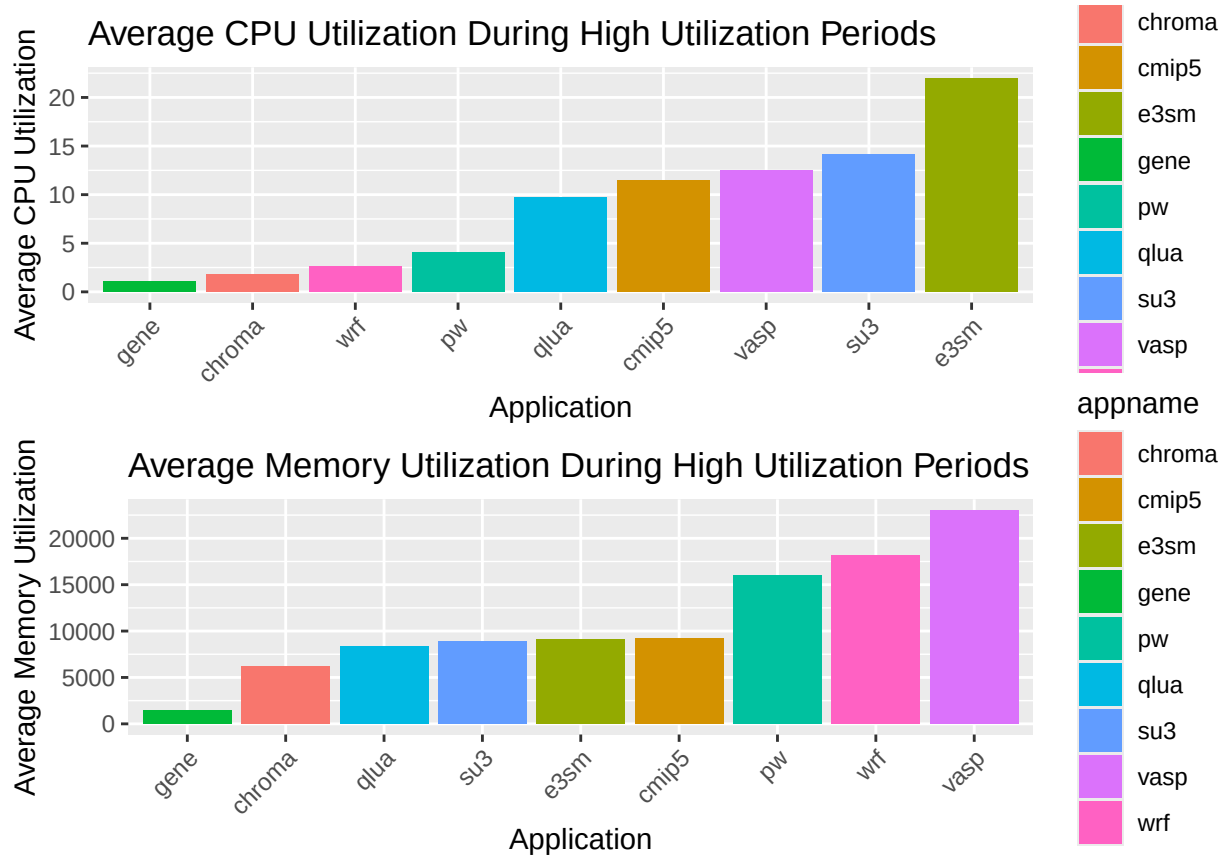
# Identifying periods with high CPU and memory utilization
high_utilization_periods <- train_data_filtered |>
  filter(cpu_utilization >= high_utilization_threshold & memory_utilization >= high_utilization_threshold)

# Calculating average resource utilization
application_usage <- high_utilization_periods |>
  group_by(appname) |>
  summarize(avg_cpu_utilization = mean(cpu_utilization),
            avg_memory_utilization = mean(memory_utilization))

# Visualising average resource utilization
plot_cpu <- ggplot(application_usage, aes(x = reorder(appname, avg_cpu_utilization), y = avg_cpu_utilization)) +
  geom_bar(stat = "identity") +
  labs(title = "Average CPU Utilization During High Utilization Periods", x = "Application", y = "Average CPU Utilization") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

plot_memory <- ggplot(application_usage, aes(x = reorder(appname, avg_memory_utilization), y = avg_memory_utilization)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Memory Utilization During High Utilization Periods", x = "Application", y = "Average Memory Utilization") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
# Arranging the plots in a single column
plot_grid(plot_cpu, plot_memory, ncol = 1)
```



High-Utilization Applications:

Top performers during intensive periods:

- vasp
- su 3
- pw
- wrf
- qlua
- e3sm

STRATEGIES:

- *Application optimization:* Applications with consistently high resource utilization during intensive periods could benefit from code optimizations, algorithmic improvements, or alternative approaches to reduce their resource usage.
- *Resource allocation and job scheduling:* The Applications with high resource demands could be allocated dedicated resource pools or scheduled in a manner that minimizes resource contention and improves overall system performance.
- *Load balancing:* Implementing load balancing techniques to distribute resource intensive workloads across multiple nodes or clusters can help lessen the periods of high resource utilization on the Cori supercomputer.

- *Capacity planning:* Identifying recurring patterns of high resource utilization can inform capacity planning decisions, such as hardware upgrades, resource allocation & distribution, or the deployment of additional computational resources.

Conclusion

Does the analysis answer the research questions?

The analysis addresses research questions by examining variations in computational loads throughout the week on Cori, identifying optimal regression models for precise predictions, understanding data relationships for future power consumption forecasts, uncovering correlations and multicollinearity, and developing strategies based on resource intensive periods.

Additionally, it identifies applications during the resource intensive periods influencing resource consumption in terms of power, memory usage, and computational load in large-scale computing environments like Cori. Therefore, the analysis answers all the research questions with best model for predicting.

Discuss the scope and generalizability of the analysis.

Scope:

The analysis comprehensively explores the computational load on Cori, a supercomputer at NERSC, by utilizing descriptive statistics, predictive modeling, correlation analysis, and resource utilization strategies. It aims to understand usage patterns, forecast power consumption, identify variable correlations, and optimize resource management. The study successfully identifies critical trends specific to Cori and provides actionable insights to improve operational efficiency and resource allocation.

Generalizability:

The generalizability of this analysis is limited due to its focus on the NERSC Cori supercomputer dataset, which may not directly translate to other computing environments. However, the employed methodologies—including predictive modeling, correlation analysis, and resource strategies—can be adapted for similar high-performance computing contexts. To enhance generalizability, incorporating more diverse datasets, addressing issues like negative memory consumption, and verifying the absence of multicollinearity would help make findings more broadly applicable.

Discuss potential limitations and possibilities for improvement.

Limitations:

- The dataset spans only one week, limiting the depth of analysis for a complex system like Cori.
- Multicollinearity poses a challenge, potentially skewing results.
- There are few observations with negative memory consumption that could be because of erroneous data or unaccounted/other factors.

Possibilities for Improvement:

- Utilizing larger datasets, such as monthly or yearly data, would offer deeper insights into trends and patterns on Cori.
- Consistent check against multicollinearity is crucial for more accurate results.
- Addressing the above instances of negative memory consumption requires specialized domain knowledge and expertise about NERSC Cori.